

Final Project Report

Group Members: Lirong Yao, Laurence Yang, Haoyu Yan, Yaxi Zeng

GitHub: <https://github.com/Yao-Lirong/CS5782>

1. Introduction

This project focuses on the re-implementation and evaluation of *LoRA: Low-Rank Adaptation of Large Language Models* by Edward J. Hu, et al. The paper addresses an important problem in adapting large pre-trained language models: as models scale to hundreds of millions or billions of parameters, full fine-tuning becomes increasingly expensive because it requires updating and storing a complete copy of all model weights for each downstream task.

LoRA proposes a parameter-efficient fine-tuning method that freezes the original pre-trained model weights and injects small trainable low-rank matrices into selected Transformer weight matrices. Instead of updating the full weight matrix W_0 , LoRA represents the update as a low-rank decomposition, where W_0 remains frozen, while A and B are trainable low-rank matrices.

$$h = W_0x + \Delta Wx = W_0x + BAx$$

Key contributions:

- **Efficiency:** Reducing the number of trainable parameters by up to 10,000 times and GPU memory requirements by 3 times compared to full fine-tuning.
- **Performance:** Maintaining or exceeding the performance of full fine-tuning on models like RoBERTa, DeBERTa, and GPT-2/3.
- **Zero Latency:** Because the low-rank matrices can be merged with the frozen weights during deployment, LoRA introduces no additional inference latency.

2. Chosen Result

(1) Efficiency-performance:

- **Natural Language Understanding:** Following *LoRA Paper Table 2*, we evaluated the accuracy of LoRA with RoBERTa-base on two GLUE tasks: SST-2 (sentiment analysis) and MRPC (paraphrase detection).
- **Natural Language Generation:** Following *LoRA Paper Table 3*, we evaluated LoRA with GPT-2 Medium on the E2E NLG Challenge, a data-to-text generation dataset. Performance was evaluated through BLEU, measuring correspondence between our model's output and a human reference.
- **Reason:** These experiments test LoRA's central claim that it can approach full fine-tuning performance while updating far fewer parameters, across both classification and generation tasks.

(2) Rank sensitivity:

- **Result:** Following *LoRA Paper Table 5*, we conducted a rank and target-module sensitivity analysis, hoping to observe that given the same number of trainable parameters, applying LoRA to both query and value projections can outperform applying it to only one projection with a higher rank.
- **Reason:** This experiment introduced a standard of how and where to apply LoRA in real-life engineering settings

3. Methodology

We designed our experiments based on practical concerns regarding compute constraints and the project's scope. The original paper used a cluster of NVIDIA Tesla V100s, but we only have one single A100 on Google Colab. Therefore, we focused on a representative subset of experiments and models of reasonable size: two GLUE classification tasks (SST-2 and MRPC) on RoBERTa-base, E2E NLG Challenge on GPT-2 Middle for generation, and a smaller rank/target-module sensitivity sweep on

RoBERTa. This setting allows us to train one epoch within 5 minutes and limit our project to a total 24–28 hour training window. We exactly replicate the paper’s hyper-parameter settings. For NLU tasks, we follow *LoRA Paper Table 9*. For NLG, we follow *LoRA Paper Table 11*.

For the sensitivity analysis, we tested LoRA on different target modules: Q-only, K-only, V-only, O-only, Q+K, Q+V, and Q+K+V+O. We controlled the LoRA parameter budget to be the same so that performance reflected target-module choice rather than parameter count. The original paper reached the solution based on a decoder-only model, GPT-3. We used RoBERTa and GLUE to test whether the same finding also applies to an encoder-only architecture.

4. Results and Analysis

Our RoBERTa results are reported in *Table 1*, which generally support the paper’s conclusion. On both tasks, our implementation achieved similar performance with slight degeneration compared to original paper. But this is still good enough to suggest that LoRA can achieve strong performance while greatly reducing training cost. The GPT-2 NLG replication, reported in *Table 2*, was more difficult. We did not fully match the paper’s BLEU score, which may be attributed to the fact that single-run result can vary due to stochasticity. (In one run, we accidentally achieved 71.2 for GPT-2 FT, outperforming paper’s 68.2).

The rank sensitivity experiment was especially informative. In our RoBERTa SST-2 setting, Q+V performed best, while Q+K+V+O was second best (*Figure 1*). This demonstrates the paper’s claim - applying LoRA to Q+V and Q+K+V+O projections perform the best - holds across model architecture.

Overall, our reproduction confirms LoRA’s qualitative contribution: low-rank adaptation can achieve competitive performance with far fewer trainable parameters. We also verified important LoRA implementation practices that given the same number of parameters, applying LoRA to both query and value projections outperform other settings.

5. Reflections

A major lesson from this project is that paper replication isn’t only about building the same model architecture, but also keeping small experiment details, like hyper-parameters, the same. Our first minimal viable implementation verified the basic pipeline, but the results were not comparable to the paper. We improved our results by a huge margin by exactly replicating the paper’s hyper-parameter settings. As a result, our SST-2 accuracy improved from about 93.5 to 95.1, and MRPC improved from about 82.8 to 88.7.

We also learned that we should always understand what a metric actually measures, not just what we assume it measures. When our BLEU scores seemed off, printing output-target pairs revealed we were computing sentence-level BLEU instead of corpus BLEU. Since BLEU measures word overlap, not semantic similarity, the mismatch was subtle. Switching to corpus BLEU with grouped references made our evaluation significantly more faithful from 29.5 to 66.2.

In addition to verifying rank-sensitivity on encoder models, we also tried to verify it on a scaled-down decoder model – GPT-2 Medium. However, we were unable to produce reliable results. We believe this was due to a sub-optimal experiment setup because this experiment was not included in the original paper and we had to design our own hyper-parameter combination. A potential future work is to run a Bayesian hyper-parameter optimization for this experiment.

If permitted more time in general, we will also run all of our experiments with multiple initial seeds and report results in confidence intervals like the paper did.

Overall, this project helped us understand both the practical value of parameter-efficient fine-tuning and the importance of careful experimental setup for exact reproducibility of deep learning results.

6. References

- Hu, Edward J., Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Liang Wang, and Weizhu Chen. "Lora: Low-rank adaptation of large language models." *Iclr* 1, no. 2 (2022): 3.
- Wang, Alex, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel Bowman. "GLUE: A multi-task benchmark and analysis platform for natural language understanding." In *Proceedings of the 2018 EMNLP workshop BlackboxNLP: Analyzing and interpreting neural networks for NLP*, pp. 353-355. 2018.
- Papineni, Kishore, Salim Roukos, Todd Ward, and Wei-Jing Zhu. "Bleu: a method for automatic evaluation of machine translation." In *Proceedings of the 40th annual meeting of the Association for Computational Linguistics*, pp. 311-318. 2002.
- Radford, Alec, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. *Language models are unsupervised multitask learners (2019)*. 2022.
- Liu, Yinhan, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. "Roberta: A robustly optimized bert pretraining approach." *arXiv preprint arXiv:1907.11692* (2019).

7. Tables and Figures

Model & Method	# Trainable Parameters	MNLI	SST-2	MRPC	CoLA	QNLI	QQP	RTE	STS-B	Avg.
RoB _{base} (FT)*	125.0M	87.6	94.8	90.2	63.6	92.8	91.9	78.7	91.2	86.4
RoB _{base} (BitFit)*	0.1M	84.7	93.7	92.7	62.0	91.8	84.0	81.5	90.8	85.2
RoB _{base} (Adpt ^D)*	0.3M	87.1 $\pm$.0	94.2 $\pm$.1	88.5 $\pm$ 1.1	60.8 $\pm$.4	93.1 $\pm$.1	90.2 $\pm$.0	71.5 $\pm$ 2.7	89.7 $\pm$.3	84.4
RoB _{base} (Adpt ^D)*	0.9M	87.3 $\pm$.1	94.7 $\pm$.3	88.4 $\pm$.1	62.6 $\pm$.9	93.0 $\pm$.2	90.6 $\pm$.0	75.9 $\pm$ 2.2	90.3 $\pm$.1	85.4
RoB _{base} (LoRA)	0.3M	87.5 $\pm$.3	95.1$\pm$.2	89.7 $\pm$.7	63.4 $\pm$ 1.2	93.3$\pm$.3	90.8 $\pm$.1	86.6$\pm$.7	91.5$\pm$.2	87.2
RoB _{large} (FT)*	355.0M	90.2	96.4	90.9	68.0	94.7	92.2	86.6	92.4	88.9
RoB _{large} (LoRA)	0.8M	90.6$\pm$.2	96.2 $\pm$.5	90.9$\pm$1.2	68.2$\pm$1.9	94.9$\pm$.3	91.6 $\pm$.1	87.4$\pm$2.5	92.6$\pm$.2	89.0
RoB _{large} (Adpt ^P)†	3.0M	90.2 $\pm$.3	96.1 $\pm$.3	90.2 $\pm$.7	68.3$\pm$1.0	94.8$\pm$.2	91.9$\pm$.1	83.8 $\pm$ 2.9	92.1 $\pm$.7	88.4
RoB _{large} (Adpt ^P)†	0.8M	90.5$\pm$.3	96.6$\pm$.2	89.7 $\pm$ 1.2	67.8 $\pm$ 2.5	94.8$\pm$.3	91.7 $\pm$.2	80.1 $\pm$ 2.9	91.9 $\pm$.4	87.9
RoB _{large} (Adpt ^H)†	6.0M	89.9 $\pm$.5	96.2 $\pm$.3	88.7 $\pm$ 2.9	66.5 $\pm$ 4.4	94.7 $\pm$.2	92.1 $\pm$.1	83.4 $\pm$ 1.1	91.0 $\pm$ 1.7	87.8
RoB _{large} (Adpt ^H)†	0.8M	90.3 $\pm$.3	96.3 $\pm$.5	87.7 $\pm$ 1.7	66.3 $\pm$ 2.0	94.7 $\pm$.2	91.5 $\pm$.1	72.9 $\pm$ 2.9	91.5 $\pm$.5	86.4
RoB _{large} (LoRA)†	0.8M	90.6$\pm$.2	96.2 $\pm$.5	90.2$\pm$1.0	68.2 $\pm$ 1.9	94.8$\pm$.3	91.6 $\pm$.2	85.2$\pm$1.1	92.3$\pm$.5	88.6
DeB _{XXL} (FT)*	1500.0M	91.8	97.2	92.0	72.0	96.0	92.7	93.9	92.9	91.1
DeB _{XXL} (LoRA)	4.7M	91.9$\pm$.2	96.9 $\pm$.2	92.6$\pm$.6	72.4$\pm$1.1	96.0$\pm$.1	92.9$\pm$.1	94.9$\pm$.4	93.0$\pm$.2	91.3

LoRA Paper Table 2 -

RoBERTa with different adaptation methods on the GLUE benchmark

Model & Method	# Trainable Parameters	E2E NLG Challenge				
		BLEU	NIST	MET	ROUGE-L	CIDEr
GPT-2 M (FT)*	354.92M	68.2	8.62	46.2	71.0	2.47
GPT-2 M (Adapter ^L)*	0.37M	66.3	8.41	45.0	69.8	2.40
GPT-2 M (Adapter ^L)*	11.09M	68.9	8.71	46.1	71.3	2.47
GPT-2 M (Adapter ^H)	11.09M	67.3 $\pm$.6	8.50 $\pm$.07	46.0 $\pm$.2	70.7 $\pm$.2	2.44 $\pm$.01
GPT-2 M (FT ^{Top2})*	25.19M	68.1	8.59	46.0	70.8	2.41
GPT-2 M (PreLayer)*	0.35M	69.7	8.81	46.1	71.4	2.49
GPT-2 M (LoRA)	0.35M	70.4$\pm$.1	8.85$\pm$.02	46.8$\pm$.2	71.8$\pm$.1	2.53$\pm$.02
GPT-2 L (FT)*	774.03M	68.5	8.78	46.0	69.9	2.45
GPT-2 L (Adapter ^L)	0.88M	69.1 $\pm$.1	8.68 $\pm$.03	46.3 $\pm$.0	71.4 $\pm$.2	2.49$\pm$.0
GPT-2 L (Adapter ^L)	23.00M	68.9 $\pm$.3	8.70 $\pm$.04	46.1 $\pm$.1	71.3 $\pm$.2	2.45 $\pm$.02
GPT-2 L (PreLayer)*	0.77M	70.3	8.85	46.2	71.7	2.47
GPT-2 L (LoRA)	0.77M	70.4$\pm$.1	8.89$\pm$.02	46.8$\pm$.2	72.0$\pm$.2	2.47 $\pm$.02

LoRA Paper Table 3 -

GPT-2 medium (M) and large (L) with different adaptation methods on the E2E NLG Challenge.

	# of Trainable Parameters = 18M						
Weight Type	W_q	W_k	W_v	W_o	W_q, W_k	W_q, W_v	W_q, W_k, W_v, W_o
Rank r	8	8	8	8	4	4	2
WikiSQL ($\pm$ 0.5%)	70.4	70.0	73.0	73.2	71.4	73.7	73.7
MultiNLI ($\pm$ 0.1%)	91.0	90.8	91.0	91.3	91.3	91.3	91.7

LoRA Paper Table 5 - Rank Sensitivity on GPT-3

Method	Dataset	MNLI	SST-2	MRPC	CoLA	QNLI	QQP	RTE	STS-B
	Optimizer	AdamW							
	Warmup Ratio	0.06							
	LR Schedule	Linear							
RoBERTa base LoRA	Batch Size	16	16	16	32	32	16	32	16
	# Epochs	30	60	30	80	25	25	80	40
	Learning Rate	5E-04	5E-04	4E-04	4E-04	4E-04	5E-04	5E-04	4E-04
	LoRA Config.	$r_q = r_v = 8$							
	LoRA α	8							
	Max Seq. Len.	512							
RoBERTa large LoRA	Batch Size	4	4	4	4	4	4	8	8
	# Epochs	10	10	20	20	10	20	20	30
	Learning Rate	3E-04	4E-04	3E-04	2E-04	2E-04	3E-04	4E-04	2E-04
	LoRA Config.	$r_q = r_v = 8$							
	LoRA α	16							
	Max Seq. Len.	128	128	512	128	512	512	512	512
RoBERTa large LoRA [†]	Batch Size	4							
	# Epochs	10	10	20	20	10	20	20	10
	Learning Rate	3E-04	4E-04	3E-04	2E-04	2E-04	3E-04	4E-04	2E-04
	LoRA Config.	$r_q = r_v = 8$							
	LoRA α	16							
	Max Seq. Len.	128							
RoBERTa large Adpt [†] (3M) [†]	Batch Size	32							
	# Epochs	10	20	20	20	10	20	20	20
	Learning Rate	3E-05	3E-05	3E-04	3E-04	3E-04	3E-04	3E-04	3E-04
	Bottleneck r	64							
	Max Seq. Len.	128							
RoBERTa large Adpt [†] (0.8M) [†]	Batch Size	32							
	# Epochs	5	20	20	20	10	20	20	20
	Learning Rate	3E-04	3E-04	3E-04	3E-04	3E-04	3E-04	3E-04	3E-04
	Bottleneck r	16							
	Max Seq. Len.	128							
RoBERTa large Adpt ^H (6M) [†]	Batch Size	32							
	# Epochs	10	5	10	10	5	20	20	10
	Learning Rate	3E-05	3E-04	3E-04	3E-04	3E-04	3E-04	3E-04	3E-04
	Bottleneck r	64							
	Max Seq. Len.	128							
RoBERTa large Adpt ^H (0.8M) [†]	Batch Size	32							
	# Epochs	10	5	10	10	5	20	20	10
	Learning Rate	3E-04	3E-04	3E-04	3E-04	3E-04	3E-04	3E-04	3E-04
	Bottleneck r	8							
	Max Seq. Len.	128							

Table 9: The hyperparameters we used for RoBERTa on the GLUE benchmark.

LoRA Paper Table 9 - Hyper-parameters used for RoBERTa on the GLUE benchmark

Dataset	E2E	WebNLG	DART
	Training		
Optimizer	AdamW		
Weight Decay	0.01	0.01	0.0
Dropout Prob	0.1	0.1	0.0
Batch Size	8		
# Epoch	5		
Warmup Steps	500		
Learning Rate Schedule	Linear		
Label Smooth	0.1	0.1	0.0
Learning Rate	0.0002		
Adaptation	$r_q = r_v = 4$		
LoRA α	32		
	Inference		
Beam Size	10		
Length Penalty	0.9	0.8	0.8
no repeat ngram size	4		

LoRA Paper Table 11 – Hyper-parameters for GPT-2 LoRA on E2E, WebNLG and DART

Model & Method	# Params	SST-2	MRPC
RoBERTa (Full FT)	125.0M	94.8	90.2
RoBERTa (BitFit)	0.1M	93.7	92.7
RoBERTa (AdapterDrop)	0.3M	94.2 $\pm$ 0.1	88.5 $\pm$ 1.1
RoBERTa (AdapterDrop)	0.9M	94.7 $\pm$ 0.3	88.4 $\pm$ 0.1
RoBERTa (LoRA paper)	0.3M	95.1$\pm$0.2	89.7 $\pm$ 0.7
RoBERTa (our LoRA)	0.3M	95.1	88.7

Table 1. RoBERTa GLUE reproduction results.

Model & Method	# Params	BLUE
GPT-2 M (Full FT)	354.92M	68.2
GPT-2 M (AdapterL)	0.37M	66.3
GPT-2 M (AdapterH)	11.09M	67.3 $\pm$ 0.6
GPT-2 M (FT top2)	25.19M	68.1
GPT-2 M (PreLayer)0	0.35	69.7
GPT-2 M (LoRA paper)	0.35M	70.4 $\pm$0.1
GPT-2 M (our LoRA)	0.35M	66.2

Table 2. GPT-2 E2E NLG reproduction results.

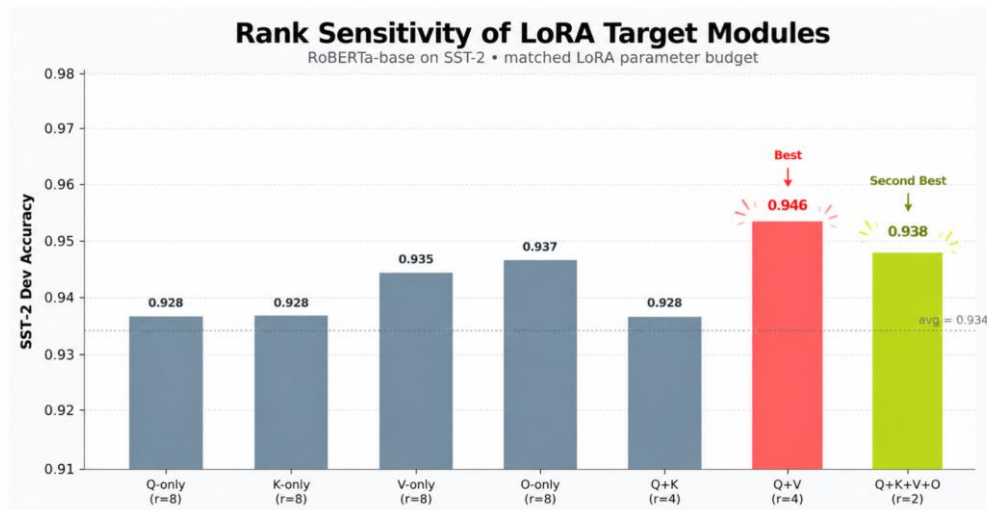


Figure 1. Rank sensitivity of LoRA target modules.